



Trailhead



[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



Write SOSL Queries

Write SOSL Queries

Learning Objectives

After completing this unit, you'll be able to:

- Understand how SOSL differs from other full-text search alternatives.
- Identify the basic SOSL syntax.
- Describe the differences between SOSL and SOQL.
- Build a SOSL query that searches across multiple sObjects.

What Is SOSL?

In the last unit you were introduced to SOQL, and you learned how you can use it to query data in sObjects and their related tables. To perform text-based queries across multiple sObjects, you can use SOSL (Salesforce Object Search Language), Salesforce's option for full-text searching.

As a .NET developer, you might be familiar with the full-text searching capabilities available with Microsoft SQL Server (MS FTS). You might also be familiar with a popular search engine library called Lucene.Net, which was ported from the original Java version to C#.

The good and bad news here is that SOSL is different from those full-text search alternatives. The biggest differences have to do with what is required to set up and maintain the indexes. Salesforce uses the open-source search and indexing engine Lucene to power SOSL, but it's been set up so that you don't have to worry about installing and configuring anything. You're also not responsible for maintaining indexes. In fact, for the most part, you don't have much control over how search indexes are used, which is good news because it means that writing SOSL queries is easy.

However, as a .NET developer you love to tweak and fine-tune your code. So hearing out that you have little control over the configuration and indexes might make you feel uncomfortable. We want to reassure you that even though you have less control, , 



Building a SOSL Query

If you're familiar with MS FTS, you know that you need to write SELECT queries that use a `CONTAINS` or `FREETEXT` statement. The powerful `CONTAINS` search, which has many variations, allows you to search for exact or fuzzy matches, as well as search for words that are close to another word.

Write SOSL Queries

SOSL uses a simpler search syntax that doesn't use the SOQL `SELECT` keyword. Instead, it uses the `FIND` keyword. A basic query looks like the following:

```
1  FIND {"grand*"} IN ALL FIELDS RETURNING Account(Name),  
    Contact(LastName, FirstName, Email)
```



Breaking this down, we see that it has three parts.

FIND Clause with Search Term

The `FIND` clause is required. It's what makes the SOSL search unique. `FIND` is followed by whatever search term you're looking for, whether it's a single word or phrase. In this case, it's the word "grand*". You can also include wildcard characters, which our example does, such as these two:

- `*` matches zero or more characters at the middle or end of the search term
- `?` matches only one character at the middle or end of the search term

We talk more about wildcards in the next unit. But in the meantime, be aware that just because you can use a wildcard in both the middle and end of the search term, doesn't mean you should. In fact, as a best practice, it's not a good idea, because it can seriously impact query performance.

IN Clause

Use the `IN` clause to specify a search group. It tells Salesforce which fields to search. But we have a really big thing that you need to be aware of here, so bear with us a bit.

In the example above, we specified `ALL FIELDS`. One might take this literally and think that all fields will be searched, but depending on which object you specify in the `RETURNING` clause, only certain text-based fields are included. Say what?



of crazy non-text based fields, it makes sense to only search through the name, email, phone, and sidebar fields. By the way, this is the default behavior.

[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

So let's say that instead of specifying **ALL FIELDS**, you want to search only name fields.

Then you would use **NAME FIELDS**. To search only phone fields, use **PHONE FIELDS**. See

[Write SOSL Queries](#)

RETURNING Clause

You use the **RETURNING** clause to specify which data to return and which objects to search. In the example above, we returned the Name field from Account and the Lastname, FirstName and Email fields from Contact. The search looked through all searchable fields on both objects, but the returned results only include those fields listed in parenthesis.

If you specify an object name without a field name in parenthesis, the search return only the ID for that object if a match is found.

The SOSL syntax has other optional clauses that you might be interested in, Check out the [official docs \(https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_sosl_syntax.htm\)](https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/sforce_api_calls_sosl_syntax.htm) to learn more about them.

Ok, so now you're wondering how does SOSL compare to other full-text search engines when it comes to fuzzy matches. Other than the **LIKE** keyword that you can use in a SOQL query and the wildcards that you can use in the SOSL query, Salesforce provides only a synonym search to look for nicknames.



The big thing to know about the nickname search is that it applies only to English-language searches on the Account, Contact, Lead, and User objects.

SOQL or SOSL?

Which one do you use? I'm sure you won't be surprised to hear this: It depends on what you're trying to accomplish.



resides in, and you need to search across multiple objects. This is especially true when those objects aren't related because SOQL works only with related objects.

[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Execute a SOSL Search

Write SOSL Queries

So now that you know why and how to use a SOSL search, let's go ahead and try building one.

You can execute a SOSL search in more than one way, but right now we're going to focus on the easiest one, which is using Query Editor in Developer Console. You will probably remember this from the last unit on SOQL.

Prerequisites

Before running the SOSL search, we need to add data to the development org. First, we'll create a sample document that we can upload to our development org.

1. In Notepad or any text-based editor, create a file and enter the following text:

```
First quarter figures were better than expected for new employee Joseph Smith. copy
```

2. Save the file to your local machine with the name `TestDocument.txt`.
3. In your Developer Edition org, on the Files tab, click **Upload Files**.
4. Browse to the location where you saved the `TestDocument.txt` file and click **Open**.
5. Click **Done**.



SOSL indexing is done asynchronously, so it's possible to make a change to the system and not see the results in your SOSL search immediately.

We also need to add data to some of our standard objects.

1. From the Setup menu, select **Developer Console**.
2. In the Developer Console, select **Debug** and then select **Open Execute Anonymous Window**.
3. Delete any existing code, and insert the following snippet:

```
1 // Add Account and related Contact
2 Account acct = new Account(
3     Name='Test Account',
4     Phone='(225) 555-8989',
5     NumberOfEmployees=10,
```





```

11 Contact con = new Contact (
12     FirstName='Joseph',
13     LastName='Smith',
14     Email='jsmith@testaccount.com',
15     AccountId=acctID);
16 insert con;

```

Database & .NET Basics (/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries

4. Click **Execute**.

Use Developer Console

Now that we have sample data to search for, we can use the Query Editor to execute a new SOSL search.

1. In Developer Console, click the **Query Editor** tab.
2. Enter the following query:

```
FIND {joey} IN ALL FIELDS RETURNING Account(Name), Contact(LastName, FirstName),
ContentVersion(Title) copy
```

First thing you notice here is that the single quotes that surrounded the search term in our earlier example have been replaced with curly brackets. We must use these instead of quotes when the SOSL search is done in the Query Editor. If the search was done in Apex code, you would use single quotes. Also notice that the search term is the word “joey,” which is a known nickname for Joseph (the actual name we’re searching for is Joseph).

3. Click **Execute**.

The search results show only one tab. The Contact that you inserted is listed under the Contact tab because it was able to find a match based on the nickname feature. But the TestDocument you uploaded isn’t listed because the nickname search applies only to Account, Contact, Lead, and User objects. Let’s revise our test, and get back all the results.

4. Replace the existing query in Query Editor with the following:

```
FIND {jos*} IN ALL FIELDS RETURNING Account(Name), Contact(LastName, FirstName),
ContentVersion(Title) copy
```

5. Click **Execute**.

6. Your search results now display two tabs. The Document tab displays the name of the document you uploaded. Notice that we used a wildcard search to get back all



Remember, Salesforce is a multi-tenanted environment. If every customer was allowed to max out the resources, the whole system would crash in no time. Searches that exceed 20,000 characters can tax the system. Therefore, searches are limited in SOSL just like they are in SOQL. Next, we're going to learn how to optimize your search queries.

Write SOSL Queries

Quiz

To complete this unit, you need to answer all the quiz questions correctly.

+100 Points

1 If you use SOSL to perform text-based queries across multiple sObjects, you must:

- ☐ A Install and configure Lucene
- ☐ B Maintain indexes
- ☐ C Define ACLs and roles
- ☐ D None of the above

2 Which one of the following is NOT part of the SOSL syntax?

- ☐ A FIND clause



[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Write SOSL Queries

3 When would you typically use SOSL over SOQL?

- ☐ A If you need data from a single object, and you know the criteria for that object
- ☐ B If the data has been recently inserted and not yet indexed
- ☐ C When you want to return all records regardless of execution governors and limits
- ☐ D If you do not know the exact fields and objects in which your data resides

4 Which of the following searches specific fields for 'cloudy' returning Contact and Lead records?

- ☐ A FIND {cloudy} IN NAME FIELDS RETURNING Contact(LastName, FirstName, Email), Lead(Name, Email)
- ☐ B FIND {cloudy} IN NAME FIELDS RETURNING Contact.LastName, Contact.FirstName, Contact.Email, Lead.NameName, Lead.Email
- ☐ C FIND {cloudy} IN TEXT FIELDS RETURNING Contact.LastName, Contact.FirstName, Contact.Email, Lead.NameName, Lead.Email



Second attempt earns 50 points. Three or more earns 25 points.

[Database & .NET Basics \(/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](/content/learn/modules/database_basics_dotnet?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOSL Queries

Learn

- Badges
- Trails
- Trailmixes
- Superbadges
- Trailhead Academy
- Career Paths

Certifications

- Certifications
- Maintain Certifications
- Verify Certifications

Community

- Trailblazer Community
- Events
- Agentblazers
- Salesforce Developers
- Salesforce Admins
- Code of Conduct
- Report Illegal Content (EU)

Extras

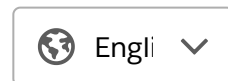
- Trailhead Login
- Sales Enablement
- Trailblazer Store
- Salesforce Help

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)



Your Privacy Choices